

SCAN DETAILS

Target	Python code (120 lines)
Mode	code
Findings	24
Generated	8/23/2026, 8:22:42 PM

1. Summary

CODE QUALITY

6/10

SECURITY

4/10

24 finding(s) were identified in this scan. Higher severity items should be fixed first.

2. Severity Breakdown

Critical	6	25% of findings	
High	6	25% of findings	
Medium	6	25% of findings	
Low	6	25% of findings	

3. Findings

Critical	CVSS 9.8	SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the
#1 Line: 42 Location: app.py - login() OWASP: A03:2021 Injection CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H		
The query is built with string formatting, so attacker-controlled input changes the SQL. A quote character breaks out of the string literal - unicode test: "quotes", dash -, arrow ->, bullet -.		
Why it matters An attacker can log in as any user or dump the whole users table without knowing a password.		
Fix suggestion Use parameterised queries, e.g. cursor.execute("SELECT * FROM users WHERE name = %s", (name,)).		
VULNERABLE CODE q = f"SELECT * FROM users WHERE name = '{name}'" cursor.execute(q)		

FIXED CODE

```
cursor.execute(
    "SELECT * FROM users WHERE name = %s", (name,)
)
```

Critical**CVSS 9.8****SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**

#2 | Line: 42 | Location: app.py - login() | OWASP: A03:2021 Injection | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

The query is built with string formatting, so attacker-controlled input changes the SQL. A quote character breaks out of the string literal - unicode test: "quotes", dash -, arrow ->, bullet -.

Why it matters

An attacker can log in as any user or dump the whole users table without knowing a password.

Fix suggestion

Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.

VULNERABLE CODE

```
q = f"SELECT * FROM users WHERE name = '{name}'"
cursor.execute(q)
```

FIXED CODE

```
cursor.execute(
    "SELECT * FROM users WHERE name = %s", (name,)
)
```

Critical**CVSS 9.8****SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**

#3 | Line: 42 | Location: app.py - login() | OWASP: A03:2021 Injection | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

The query is built with string formatting, so attacker-controlled input changes the SQL. A quote character breaks out of the string literal - unicode test: "quotes", dash -, arrow ->, bullet -.

Why it matters

An attacker can log in as any user or dump the whole users table without knowing a password.

Fix suggestion

Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.

VULNERABLE CODE

```
q = f"SELECT * FROM users WHERE name = '{name}'"
cursor.execute(q)
```

FIXED CODE

```
cursor.execute(
    "SELECT * FROM users WHERE name = %s", (name,)
)
```

Critical**CVSS 9.8****SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**

#4 | Line: 42 | Location: app.py - login() | OWASP: A03:2021 Injection | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

The query is built with string formatting, so attacker-controlled input changes the SQL. A quote character breaks out of the string literal - unicode test: "quotes", dash -, arrow ->, bullet -.

Why it matters

An attacker can log in as any user or dump the whole users table without knowing a password.

Fix suggestion

Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.

VULNERABLE CODE

```
q = f"SELECT * FROM users WHERE name = '{name}'"
cursor.execute(q)
```

FIXED CODE

```
cursor.execute(
    "SELECT * FROM users WHERE name = %s", (name,)
)
```

Critical**CVSS 9.8****SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**

#5 | Line: 42 | Location: app.py - login() | OWASP: A03:2021 Injection | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

The query is built with string formatting, so attacker-controlled input changes the SQL. A quote character breaks out of the string literal - unicode test: "quotes", dash -, arrow ->, bullet -.

Why it matters

An attacker can log in as any user or dump the whole users table without knowing a password.

Fix suggestion

Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.

VULNERABLE CODE

```
q = f"SELECT * FROM users WHERE name = '{name}'"
cursor.execute(q)
```

FIXED CODE

```
cursor.execute(
    "SELECT * FROM users WHERE name = %s", (name,)
)
```

Critical**CVSS 9.8****SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**

#6 | Line: 42 | Location: app.py - login() | OWASP: A03:2021 Injection | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

The query is built with string formatting, so attacker-controlled input changes the SQL. A quote character breaks out of the string literal - unicode test: "quotes", dash -, arrow ->, bullet -.

Why it matters

An attacker can log in as any user or dump the whole users table without knowing a password.

Fix suggestion

Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.

VULNERABLE CODE

```
q = f"SELECT * FROM users WHERE name = '{name}'"
cursor.execute(q)
```

FIXED CODE

```
cursor.execute(
    "SELECT * FROM users WHERE name = %s", (name,)
)
```

High**CVSS 7.5**

Hardcoded API key

#7 | Line: 7 | OWASP: A02:2021 Cryptographic Failures

A secret token is committed in source control.

Why it matters

Anyone with repository access can use the key.

Fix suggestion

Move the key to an environment variable and rotate it.

VULNERABLE CODE

```
API_KEY = "sk-live-abcdefghijklmnop"
```

FIXED CODE

```
API_KEY = os.environ["API_KEY"]
```

High**CVSS 7.5**

Hardcoded API key

#8 | Line: 7 | OWASP: A02:2021 Cryptographic Failures

A secret token is committed in source control.

Why it matters

Anyone with repository access can use the key.

Fix suggestion

Move the key to an environment variable and rotate it.

VULNERABLE CODE

```
API_KEY = "sk-live-abcdefghijklmnop"
```

FIXED CODE

```
API_KEY = os.environ["API_KEY"]
```

High

CVSS 7.5

Hardcoded API key

#9 | Line: 7 | OWASP: A02:2021 Cryptographic Failures

A secret token is committed in source control.

Why it matters

Anyone with repository access can use the key.

Fix suggestion

Move the key to an environment variable and rotate it.

VULNERABLE CODE

```
API_KEY = "sk-live-abcdefghijklmnop"
```

FIXED CODE

```
API_KEY = os.environ["API_KEY"]
```

High

CVSS 7.5

Hardcoded API key

#10 | Line: 7 | OWASP: A02:2021 Cryptographic Failures

A secret token is committed in source control.

Why it matters

Anyone with repository access can use the key.

Fix suggestion

Move the key to an environment variable and rotate it.

VULNERABLE CODE

```
API_KEY = "sk-live-abcdefghijklmnop"
```

FIXED CODE

```
API_KEY = os.environ["API_KEY"]
```

High

CVSS 7.5

Hardcoded API key

#11 | Line: 7 | OWASP: A02:2021 Cryptographic Failures

A secret token is committed in source control.

Why it matters

Anyone with repository access can use the key.

Fix suggestion

Move the key to an environment variable and rotate it.

VULNERABLE CODE

```
API_KEY = "sk-live-abcdefghijklmnop"
```

FIXED CODE

```
API_KEY = os.environ["API_KEY"]
```

High

CVSS 7.5

Hardcoded API key

#12 | Line: 7 | OWASP: A02:2021 Cryptographic Failures

A secret token is committed in source control.

Why it matters

Anyone with repository access can use the key.

Fix suggestion

Move the key to an environment variable and rotate it.

VULNERABLE CODE

```
API_KEY = "sk-live-abcdefghijklmnop"
```

FIXED CODE

```
API_KEY = os.environ["API_KEY"]
```

Medium

CVSS 5.3

Missing input validation

#13 | Line: N/A

User input is used without length checks.

Why it matters

Very large inputs can slow the endpoint down.

Fix suggestion

Validate and cap input length before processing.

Medium

CVSS 5.3

Missing input validation

#14 | Line: N/A

User input is used without length checks.

Why it matters

Very large inputs can slow the endpoint down.

Fix suggestion

Validate and cap input length before processing.

Medium

CVSS 5.3

Missing input validation

#15 | Line: N/A

User input is used without length checks.

Why it matters

Very large inputs can slow the endpoint down.

Fix suggestion

Validate and cap input length before processing.

Medium

CVSS 5.3

Missing input validation

#16 | Line: N/A

User input is used without length checks.

Why it matters

Very large inputs can slow the endpoint down.

Fix suggestion

Validate and cap input length before processing.

Medium

CVSS 5.3

Missing input validation

#17 | Line: N/A

User input is used without length checks.

Why it matters

Very large inputs can slow the endpoint down.

Fix suggestion

Validate and cap input length before processing.

Medium

CVSS 5.3

Missing input validation

#18 | Line: N/A

User input is used without length checks.

Why it matters

Very large inputs can slow the endpoint down.

Fix suggestion

Validate and cap input length before processing.

Low

CVSS 3.1

Outdated dependency

#19 | Location: requirements.txt | OWASP: A06:2021 Vulnerable Components

An old library version is pinned.

Why it matters

Known bugs are fixed in newer releases.

Fix suggestion

Upgrade to the latest patch version.

Low

CVSS 3.1

Outdated dependency

#20 | Location: requirements.txt | OWASP: A06:2021 Vulnerable Components

An old library version is pinned.

Why it matters

Known bugs are fixed in newer releases.

Fix suggestion

Upgrade to the latest patch version.

Low

CVSS 3.1

Outdated dependency

#21 | Location: requirements.txt | OWASP: A06:2021 Vulnerable Components

An old library version is pinned.

Why it matters

Known bugs are fixed in newer releases.

Fix suggestion

Upgrade to the latest patch version.

Low

CVSS 3.1

Outdated dependency

#22 | Location: requirements.txt | OWASP: A06:2021 Vulnerable Components

An old library version is pinned.

Why it matters

Known bugs are fixed in newer releases.

Fix suggestion

Upgrade to the latest patch version.

Low

CVSS 3.1

Outdated dependency

#23 | Location: requirements.txt | OWASP: A06:2021 Vulnerable Components

An old library version is pinned.

Why it matters

Known bugs are fixed in newer releases.

Fix suggestion

Upgrade to the latest patch version.

Low

CVSS 3.1

Outdated dependency

#24 | Location: requirements.txt | OWASP: A06:2021 Vulnerable Components

An old library version is pinned.

Why it matters

Known bugs are fixed in newer releases.

Fix suggestion

Upgrade to the latest patch version.

4. Recommendations / Fixes

- 1. SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**
Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.
- 2. SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**
Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.

- 3. SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**
Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.
- 4. SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**
Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.
- 5. SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**
Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.
- 6. SQL injection in login query - very long title that must wrap across lines to test the wrapping behaviour of the**
Use parameterised queries, e.g. `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))`.
- 7. Hardcoded API key**
Move the key to an environment variable and rotate it.
- 8. Hardcoded API key**
Move the key to an environment variable and rotate it.
- 9. Hardcoded API key**
Move the key to an environment variable and rotate it.
- 10. Hardcoded API key**
Move the key to an environment variable and rotate it.
- 11. Hardcoded API key**
Move the key to an environment variable and rotate it.
- 12. Hardcoded API key**
Move the key to an environment variable and rotate it.
- 13. Missing input validation**
Validate and cap input length before processing.
- 14. Missing input validation**
Validate and cap input length before processing.
- 15. Missing input validation**
Validate and cap input length before processing.
- 16. Missing input validation**
Validate and cap input length before processing.
- 17. Missing input validation**
Validate and cap input length before processing.
- 18. Missing input validation**
Validate and cap input length before processing.
- 19. Outdated dependency**
Upgrade to the latest patch version.
- 20. Outdated dependency**
Upgrade to the latest patch version.
- 21. Outdated dependency**
Upgrade to the latest patch version.

22. Outdated dependency

Upgrade to the latest patch version.

23. Outdated dependency

Upgrade to the latest patch version.

24. Outdated dependency

Upgrade to the latest patch version.

AI-assisted analysis - verify all findings manually. Authorized testing only. Copyright (c) 2026 Dinesh. All rights reserved. | dinesh.ai